MSDS 458 Assignment 02: CNN Architecture Analysis

Kshaunish Soni

October 21, 2026

Abstract

This study evaluates Deep Neural Network (DNN) and Convolutional Neural Network (CNN) architectures for facial recognition and image classification in mobile user authentication. Across ten distinct configurations, CNN models consistently outperformed DNN models, demonstrating superior feature extraction capabilities and spatial hierarchical learning. The top-performing architecture, C04 (a 3-layer CNN with 5x5 kernels and 128 filters), achieved a peak validation accuracy of 79.68% and a test accuracy of 79.56%. While CNNs exhibited significantly higher training times influenced by kernel size and stride, feature map analysis confirms their efficacy in capturing complex grid-based visual patterns compared to fully connected architectures.

Introduction

The management problem provided is the development of a neural network model that can accurately determine the user of a mobile device using facial recognition. The assignment asks to build a variety of neural networks, utilizing both Deep Neural Network and Convolutional Neural Network architecture to build increasingly accurate models for facial recognition, image processing, and computer vision in general.

Literature Review

In order to garner an understanding of previous successful facial recognition models using CNN, I found a systematic review of CNN architectures in face recognition. [1] Computational Facial recognition has been studied since 1964 and has grown as data science and machine learning has expanded. Machine learning was first utilized in facial recognition in 2011, and has been implemented in applications across sectors such as security, law enforcement, and healthcare.

In 2016, Sharma et al. developed a facial recognition system utilizing DNN and DNN, trained on a set of 11,284 cropped grayscale photos with 96x96 resolution. It was able to achieve 96% accuracy after 20 epochs. In 2018, Qu et al. proposed a real time facial recognition model based on a CNN hosted on an FPGA, which allowed for parallel computing and unique logic circuit design. The recognition rate was “99.25% greater than the human eye.”

Overall, the researchers of the systematic review concluded that deep learning architectures like FaceNet, VGG16, and MTCNN are superior compared to models such as Support Vector Machines as they increase scalability and accuracy. There are a variety of CNN architectures that can be applied to different tasks such as low quality images and frontal face recognition. Still, there are persistent issues such as camera angle and lighting that researchers are currently attempting to solve.

[1] Nemavhola, Andisani, Colin Chibaya, and Serestina Viriri. 2025. "A Systematic Review of CNN Architectures, Databases, Performance Metrics, and Applications in Face Recognition" Information 16, no. 2: 107. https://doi.org/10.3390/info16020107

Methods

10 architectures were created, 5 with DNN architecture, and 5 with CNN architecture. I used a similar approach to assignment 1 where I created a generic model builder and passed each architecture as a configuration. The deep learning networks were relatively simple, as each image was flattened into a dense, fully connected layer. The configuration grew in complexity as the convolution models require the number of filters, the kernels, max pool size, and stride for each layer. Generally, each model used a max pool size of 2x2, but the number of filters, the kernel size, and the stride were all aspects of the convolutional neural network I wished to experiment on. Hyperparameter tuning was utilized over learning rate, batch rize, dropout, L2, and activation. Once the hyperparameters are tuned, the model is trained on an 80/20 split of training and testing data. Each model is evaluated based on validation accuracy, with the outputs put into a results folder for analysis.

Results

Table 1's final results[1] show a very clear picture of the strength of convolution in image processing. While some DNN models were able to boast decent training accuracies, D03 hitting 85.22%, all of the DNN models struggled with validation and test accuracy, especially when compared to even the simplest convolutional models. The weakest CNN, C05, had a validation accuracy of 69.08% with just 219k parameters. Notably, CNNs suffered much higher tuning and training times compared to DNNs, the longest CNN taking almost 3 hours end to end. C04 was the best model based on training, validation, and test accuracy. The learning and loss curves of all models[2] show signs of overfitting as the training accuracy was consistently higher than the validation and test accuracy, even with the addition of adam optimizer, batch normalization, and dropout.

The feature maps[3] provide an interesting depiction of the "thought process" of the CNNs. Each convolutional layer shows a filtered, highlighted version of the image before it. In both C01 and C04, there is a clear pattern with the first layer where the highlighted areas show the notable feature in each image. For example, we can clearly see an outline of the horse in the first and second layers. As we move further into the CNN, however, the features become much harder to distinguish to the human eye. It is important to note that by the end of the process, each pixel holds information on a much larger portion of the original image, which is more interpretable for the algorithm than it is for humans.

Conclusions

Based on the multitude of experiments conducted in this facial recognition project, it is clear that Convolutional Neural Networks perform better with image related data. Every single CNN outperformed every DNN, and while I am sure that certain data preprocessing can help tune DNNs to perform better, I think the CNN solution is a better overall fit. Convolution's ability to compress complex grid data into digestible, dense values allows CNNs to perform incredibly well with image data. There was notable overfitting in all cases, even with batch normalization, dropout, and Adam optimization. As dropout and L2 regularization was tuned for each model, other potential regularization techniques could be used, such as enhancing the training data and introducing global average pooling, which reduces the final dense layer into a single value.

The feature maps were an interesting depiction into the learning process of the models. Generally DNNs are considered a black box, where interpretability is very difficult at scale. But in CNN, at least the first few layers, it is clear how to model is interpreting the image and highlighting the features it deems important. As we go further into the model, is it much harder to understand the data. For example, in C04, the horse image vaguely looks like a horse for the first two convolution layers, but by the final layer there is just one blob. While this is difficult for humans to comprehend, I would never see that image and know with accuracy that it is a horse compared to a cat, machines seem to do quite well with it.

One final note regarding the training time, as CNNs produced much longer tuning and training times compared to DNNs. It seems there were two main reasons for significantly higher training times. The kernel size, and the stride. In kernel 3x3 models, training time was manageable, around 30 minutes end to end. With C05, where stride was 2, the training time was significantly shorter than the rest of the models, though it performed worse. The 5x5 kernel with 3 convolution layers took significantly longer than the 2x2 kernel with 3 convolution layers, indicating that the kernel size is one of the most important features when it comes to impact on training time, as well as accuracy.

Appendix: Github

https://github.com/KshaunishSoni123/MSDS458_KshaunishSoni.git

Appendix: Tables and Figures

Table 1: Final Results

Final results: accuracy, loss and processing time by model (best per column in bold)

Model	Type	Params	Train acc	Train loss	Val acc	Val loss	Test acc	Test loss	Tuning (s)	Training (s)	Total (s)
D01_1x128	DNN	395,018	68.31%	0.9389	53.82%	1.3511	54.43%	1.3362	83.9	31.4	115.3
D02_1x512	DNN	1,580,042	81.66%	0.5705	55.50%	1.4216	55.53%	1.4030	246.4	99.2	345.6
D03_2_512-256	DNN	1,709,578	85.22%	0.4516	57.06%	1.4028	57.61%	1.3782	268.0	91.3	359.3
D04_3_1024-512-256	DNN	3,810,826	61.17%	1.0939	51.16%	1.4270	51.52%	1.3985	513.2	86.1	599.3
D05_4_1024-512-256-128	DNN	3,842,826	60.86%	1.0886	50.04%	1.4007	51.31%	1.3743	549.7	104.7	654.4
C01_2conv_3x3_64	CNN	565,194	93.43%	0.1814	72.18%	1.0814	72.43%	1.0639	1,758.8	308.8	2,067.6
C02_3conv_3x3_64	CNN	209,098	91.99%	0.2333	76.28%	**0.7554**	77.15%	**0.7350**	1,947.9	338.8	2,286.7
C03_2conv_5x5_128	CNN	1,470,602	93.00%	0.2011	75.26%	0.8230	75.43%	0.8452	7,958.6	1,461.8	9,420.4
C04_3conv_5x5_128	CNN	1,094,282	**96.60%**	**0.0988**	**79.68%**	0.7873	**79.56%**	0.7873	8,788.8	1,626.4	10,415.2
C05_2conv_3x3_128_s2	CNN	219,274	89.35%	0.3458	69.08%	0.9220	69.92%	0.9083	643.4	185.6	829.0

Table 2: Learning Curves by Architecture

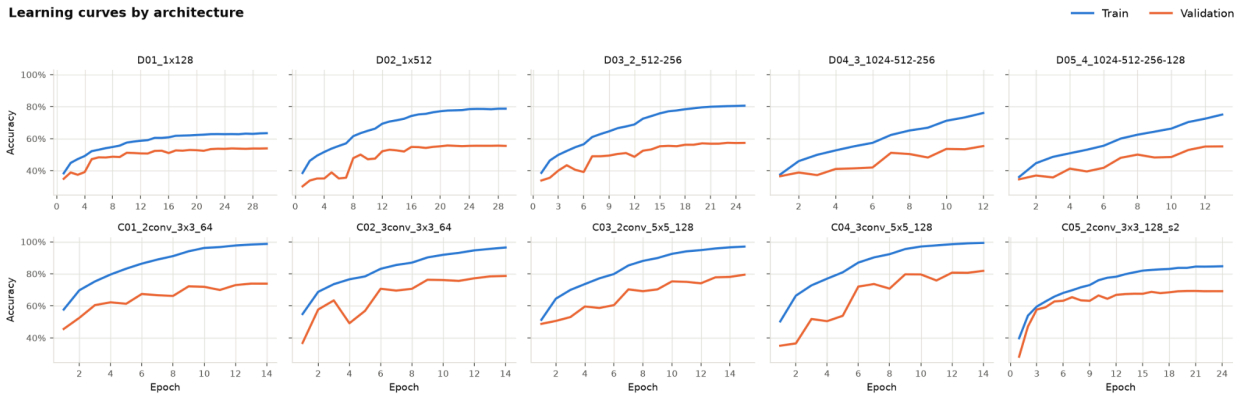


Table 3: Loss Curves by Architecture

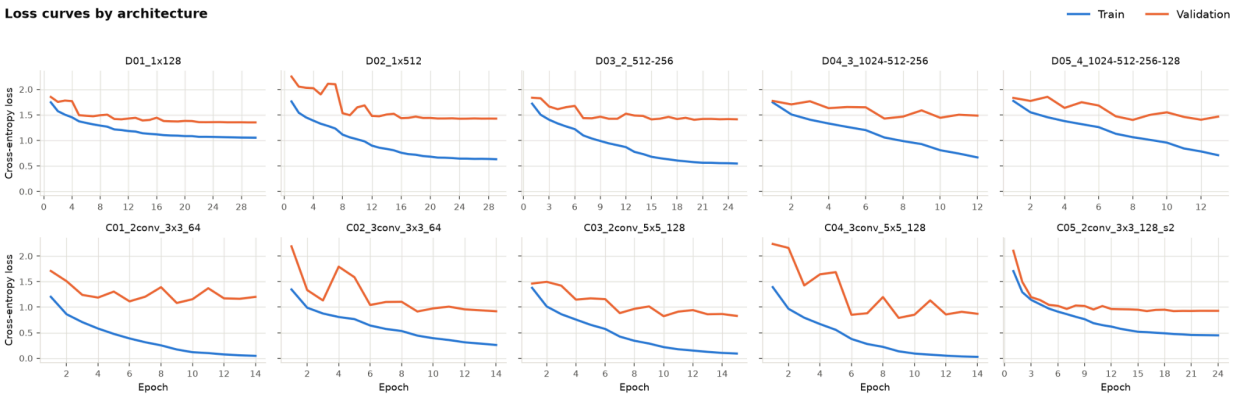


Table 4: Accuracy by Architecture

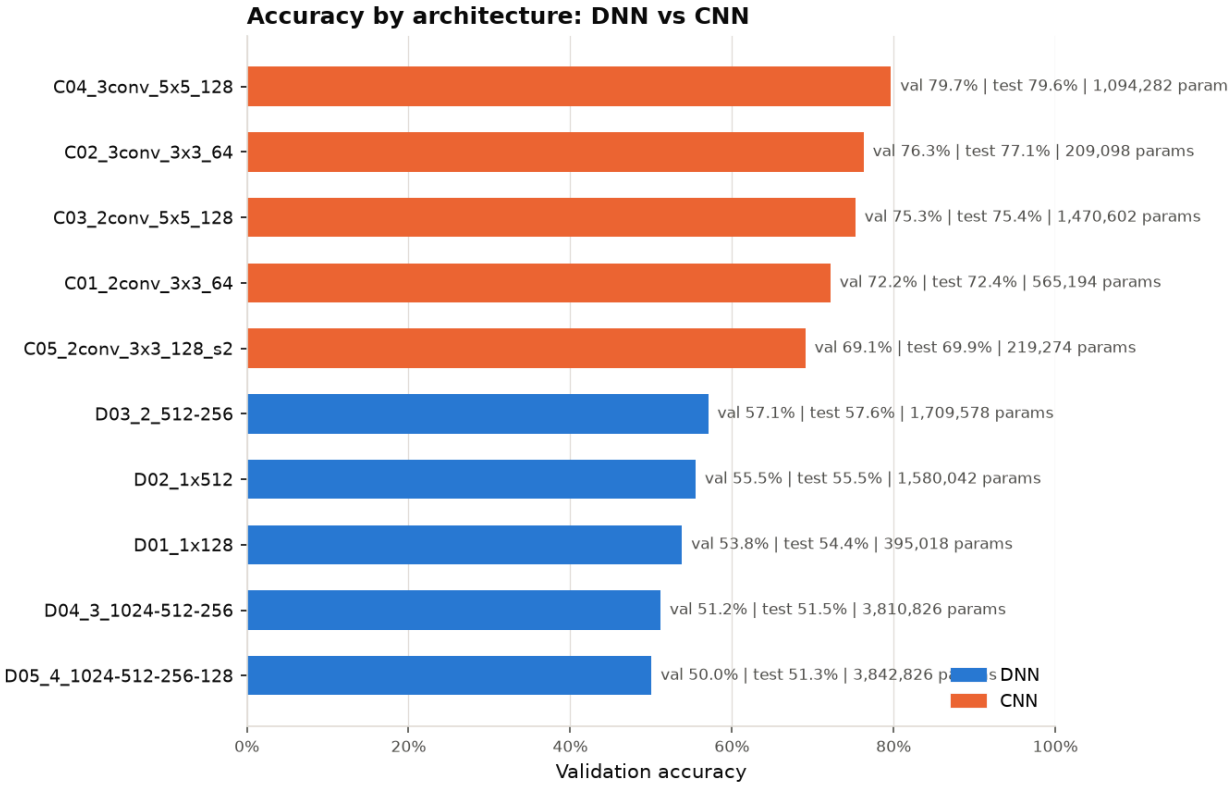


Figure 1: C04 Max-Pooling Feature Maps

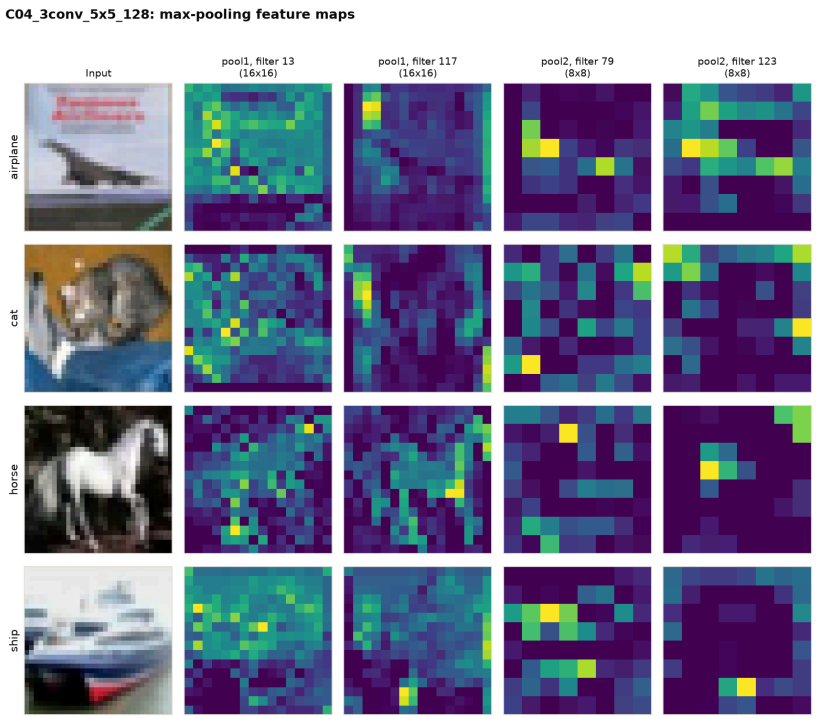
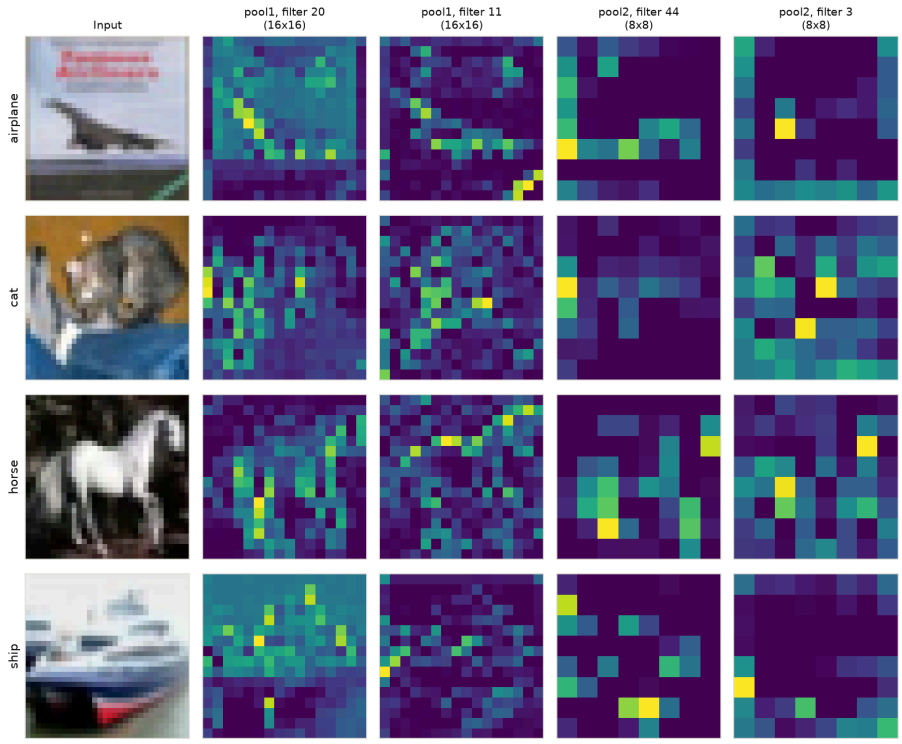


Figure 2: C01 Max-Pooling Feature Maps



Footnotes

[1] See Table 1, Table 4, and Table 2 in the Appendix for full architecture results and accuracy comparisons.

[2] See Table 2 and Table 3 in the Appendix for learning and loss curves across all model configurations.

[3] See Figure 1 and Figure 2 in the Appendix for detailed feature map visualizations of models C04 and C01.